

Overview

Context is the working memory of LLM-based systems. Poor context design causes hallucinations, contradiction, and rising cost. Context engineering is the discipline of selecting, structuring, and refreshing the right information at the right time.

In long-horizon agents, context quality often matters more than raw model size.

A simple context-budget equation:

$$B = B_{system} + B_{task} + B_{retrieval} + B_{history} + B_{tools}$$

Where total token budget B must stay below model limits while preserving relevant facts.

Context Engineering

Formal Definition

Context engineering is the set of methods that optimize what information enters the model context window, in what format, and at what moment, to maximize task success under constraints.

Core Techniques

1. Prompt structuring
 - Separate policy, task, constraints, and output schema.
2. Windowing
 - Keep only most relevant recent turns.
3. Summarization
 - Compress old interactions into stable facts.
4. Retrieval augmentation
 - Inject external evidence on demand.
5. Routing
 - Send tasks to specialized prompts/tools.

Text diagram:

User goal -> Router -> Retriever/Summarizer -> Planner -> Executor

Context Quality Heuristics

- Specificity: include concrete facts, not vague history.
- Recency: prioritize latest state for action-heavy workflows.
- Grounding: inject sources for high-stakes answers.
- Minimality: avoid irrelevant token noise.

Memory Types

Short-Term Memory

Session-local working memory.

Examples:

- recent dialogue turns
- current task state
- temporary tool outputs

Long-Term Memory

Persistent memory stores across sessions.

Examples:

- vector embeddings for documents
- profile attributes
- historical decisions

Episodic vs Semantic Memory

- Episodic memory: what happened in a specific interaction.
- Semantic memory: stable facts distilled from many interactions.

Practical architecture uses both:

- episodic logs for traceability
- semantic summaries for efficient reuse

Planning Approaches

Task Decomposition

Break goal into ordered subtasks with dependencies.

Algorithmic methods:

- deterministic planning templates
- rule-based state machines

LLM-based methods:

- free-form plan generation
- tool-selection reasoning

Plan-Execute-Reflect

1. Generate plan.
2. Execute step.
3. Evaluate step output.
4. Re-plan if needed.

This loop improves resilience in uncertain environments.

Re-Planning Triggers

- tool failure
- low confidence output
- missing required evidence
- changed user intent

Tools & Patterns

RAG as Memory Extension

RAG functions as external memory:

- index knowledge chunks

- retrieve relevant chunks per step
- augment prompt with citations

Framework Patterns

- LangGraph state + checkpointing for long workflows.
- LangChain memory abstractions for conversation context.
- Custom memory layers (Redis/vector DB) for production control.

Common Pitfalls

1. Context overflow
 - critical instructions truncated silently.
2. Summary drift
 - inaccurate summaries compound over time.
3. Memory contamination
 - unrelated documents pulled into prompt.
4. Planning brittleness
 - plan fails when environment changes.

Mitigations:

- explicit token budgeting
- summary validation checkpoints
- retrieval quality thresholds
- fallback deterministic subflows

Business Case Studies & Exceptions

Case Study 1: CRM Copilot with Long-Term Context

A sales copilot stores account history and recent interactions. It retrieves relevant deals, objections, and previous commitments before drafting outreach.

Outcome:

- improved continuity across account managers
- reduced repeated discovery questions

Exception:

- strict data retention policies can limit long-term memory storage.

Case Study 2: Travel Planning Agent Failure

A travel assistant planned flights and hotels but failed when cancellation policies changed mid-workflow.

Fix:

- re-planning trigger on policy uncertainty
- mandatory retrieval refresh before booking actions

Lesson:

Planning must be dynamic; static plans fail in changing environments.

Interview Questions & Answers

1. **What is context engineering?**
Designing what information enters model context, when, and in what format to improve outcomes.
2. **Why is context engineering critical for agents?**
Agents run over many steps; poor context management leads to drift and failure.
3. **Difference between short-term and long-term memory?**
Short-term is session-local; long-term persists across sessions.
4. **What is episodic memory in AI agents?**
Memory of specific past events/interactions.
5. **What is semantic memory?**
Generalized stable facts extracted from past interactions.
6. **How does RAG support memory?**
It retrieves relevant external knowledge on demand instead of storing everything in prompt.
7. **What causes context overflow?**
Unbounded chat history, large retrieval payloads, and verbose tool outputs.
8. **How do you handle long conversations?**
Summarize old turns, keep recent critical turns, and retrieve details as needed.
9. **What is re-planning?**
Updating task plan during execution based on new evidence or failures.
10. **When should planning be deterministic?**
For safety-critical or compliance-critical substeps.
11. **How do you validate summary quality?**
Compare summary facts with source excerpts and run consistency checks.
12. **What metrics assess memory quality?**
Recall accuracy, contradiction rate, and retrieval precision.
13. **What is label leakage analog in context systems?**
Injecting future or unauthorized information that should not be used for decision-making.
14. **How can routing improve context quality?**
By sending tasks to specialized prompts/tools and limiting irrelevant context.
15. **What is the main production risk in context-heavy systems?**
Silent degradation due to stale summaries and weak retrieval relevance.

References

- Anthropic context engineering guide: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Anthropic effective agents guide: <https://resources.anthropic.com/building-effective-ai-agents>
- NYU Stern course page: <https://aiagents.stern.nyu.edu/>
- LangGraph docs: <https://docs.langchain.com/oss/python/langgraph/graph-api>
- DeepLearning.AI Agentic AI course page: <https://www.deeplearning.ai/courses/agent-ai>
- IITM Pravartak context engineering program page: <https://futureense.com/iitm-pravartak/advanced-engineering-program-in-applied-ai-ml-with-context-engineering>